SUP'COM

Traitement d'images médicales

Enseignante : Mme Wafa Rekik

Atelier 6 : plateforme web d'analyse d'images médicales

Réalisé par : Oussema Harrabi - INDP3 - SIA 26-27

30 octobre 2026

Lumen réunit les cinq ateliers du module dans une seule application web.
Le navigateur affiche l'image au pixel, l'API FastAPI exécute des pipelines d'étapes.
Cinq chaînes retrouvent les notes du cours, de 0,086 à 0,959, et le recalage tombe à 2,87 px.

1 Objectif et périmètre

L'énoncé demande une application web qui prenne une image médicale, la prétraite, la segmente par des méthodes conventionnelles puis par un réseau pré-entraîné, recale deux images, et permette de comparer les résultats chiffrés dans la même interface.

Lumen répond à cette demande en rassemblant les cinq ateliers précédents dans un seul outil. Chaque brique du module devient un module de la plateforme, avec ses méthodes et son atelier d'origine.

TABLEAU 1. MODULES DE LA PLATEFORME

MODULE	MÉTHODES	ATELIER
Ingestion	import image, extraction de séquence, métadonnées	1
Prétraitement	médian, gaussien, moyenneur, CLAHE, niveaux de gris	2
Segmentation conventionnelle	seuil manuel, Otsu, k-means, MeanShift, watershed, Chan-Vese, morphologie	2 et 3
Contours	Sobel, Laplacien, gradient morphologique, superposition	2
Segmentation profonde	U-Net ONNX, jeu endoscopie et jeu BUSI	4
Recalage	translation, affine, homographie, flot optique, NCC, MSE, NMI, landmarks	5
Évaluation	Dice, IoU, rappel, précision, durée par étape	3 et 4
Comparaison	grille de résultats, curseur, tableau de mesures	6

Deux publics : l'étudiant qui prépare ses comptes rendus et veut rejouer un TP en retrouvant les scores du cours, et l'enseignante en soutenance, qui doit vérifier en trois clics que chaque méthode annoncée tourne vraiment.

2 Architecture

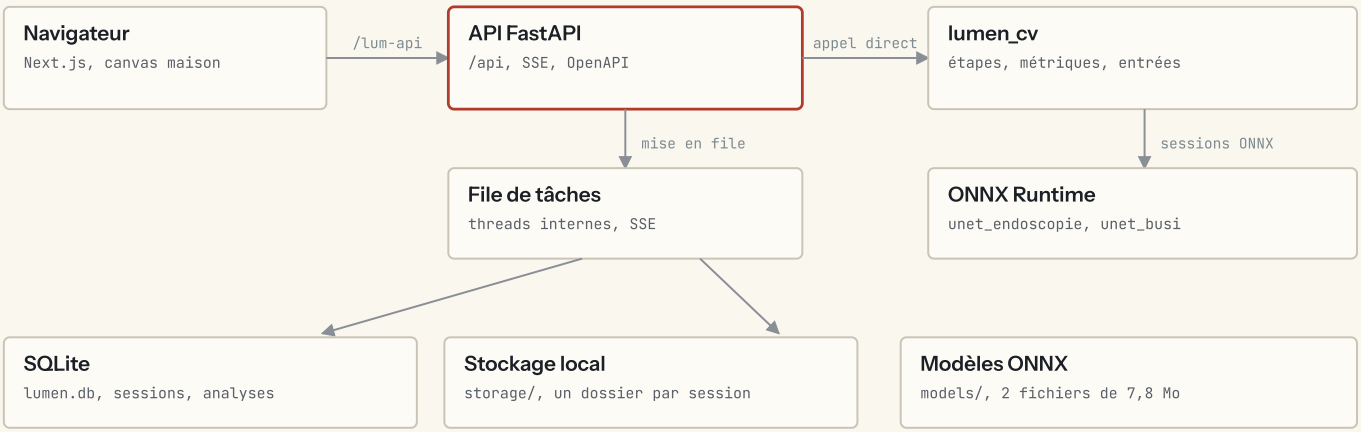


FIGURE 1 Six blocs et un seul sens de dépendance : le navigateur appelle le proxy Next.js, qui appelle l'API, qui appelle lumen_cv ; le stockage et les modèles restent sur la machine.

Le navigateur ne connaît qu'une seule origine. Next.js renvoie tout ce qui commence par `/lum-api` vers FastAPI, ce qui garde le cookie de session et permet de relire les pixels du masque dans un canvas sans négociation CORS.

L'API ne calcule rien elle-même : elle valide les étapes, écrit l'analyse en base, la met en file et appelle `lumen_cv`. Le paquet de traitement ignore FastAPI et SQLAlchemy, donc il se teste seul, sans serveur ni navigateur.

Les deux modèles U-Net sont convertis une fois en ONNX et versionnés dans `models/`. L'inférence se fait sur le processeur, dans le même processus que les méthodes classiques.

TABLEAU 2. PILE TECHNIQUE ET RAISON DU CHOIX

BRIQUE	CHOIX	RAISON
Interface	Next.js 16 (App Router), canvas maison	le masque se pose au pixel, le zoom et la lecture de valeur au curseur demandent un contexte 2D, pas une balise img
Contrat	OpenAPI généré par FastAPI	renommer un paramètre casse les types TypeScript au lieu de casser l'interface en silence
Traitement	lumen_cv (OpenCV, scikit-image, scikit-learn)	code pur, sans import web, 41 tests qui tournent sans lancer de serveur
Inférence	ONNX Runtime, deux U-Net	8 Mo de modèles au lieu de 350 Mo de roue TensorFlow, pour la même sortie
Tâches	file interne à threads, suivi SSE	le calcul survit à la fermeture de l'onglet et la progression arrive sans sondage

3 Chaîne de traitement et API

Une analyse est une liste ordonnée d'étapes. Chaque étape a un type et des paramètres, et le catalogue publié par `GET /api/schema/pipeline` donne les bornes de chaque paramètre. La chaîne watershed tient en quatre étapes.

PIPELINE DE LA CHAÎNE WATERSHED, CORPS JSON

```
{"steps": [
  {"kind": "preprocess.gray", "params": {}},
  {"kind": "segment.watershed", "params": {"marker_ratio": 0.5}},
  {"kind": "postprocess.open", "params": {"ksize": 3}},
  {"kind": "postprocess.largest_component", "params": {}}
]}
```

Le paquet exécute ces étapes dans l'ordre, chronomètre chacune et garde le dernier masque produit. Une étape en échec lève une erreur qui porte son nom, ce qui évite de chercher l'origine dans un message générique.

LUMEN_CV/PIPELINE.PY, BOUCLE D'EXÉCUTION

```
    for spec, params in prepared:
        start = perf_counter()
        try:
            if spec.apply_with_models is not None and models is not None:
                output, diagnostics = spec.apply_with_models(current, params, models)
            else:
                output, diagnostics = spec.apply(current, params)
        except ModelMissingError:
            raise
        except Exception as error:
            raise StepError(spec.kind, str(error)) from error
        duration_ms = (perf_counter() - start) * 1000.0
```

Côté API, la création d'analyse valide les étapes avant d'écrire quoi que ce soit, puis rend la main tout de suite avec un code 202 et un identifiant de tâche.

ANALYSES.PY, FIN DE CREATE_ANALYSIS

```
        status="queued",
        progress=0.0,
    )
    db.add(analysis)
    db.commit()
    db.refresh(analysis)
    request.app.state.jobs.submit(runner.execute_analysis, analysis.id)
    return AnalysisOut.from_row(analysis)
```

L'API expose vingt routes, vingt-quatre opérations avec les méthodes HTTP, toutes décrites par le contrat OpenAPI généré au démarrage. Le front en dérive ses types au lieu de recopier les noms de champs, et le schéma du pipeline alimente directement les formulaires de l'inspecteur.

TABLEAU 3. ROUTES EXPOSÉES, PAR FAMILLE

FAMILLE	ROUTES	OPÉRATIONS
Analyses	5	7
Ressources	3	5
Recalage	5	5
Service	4	4
Démonstrations	3	3
Total	20	24

4 Interface

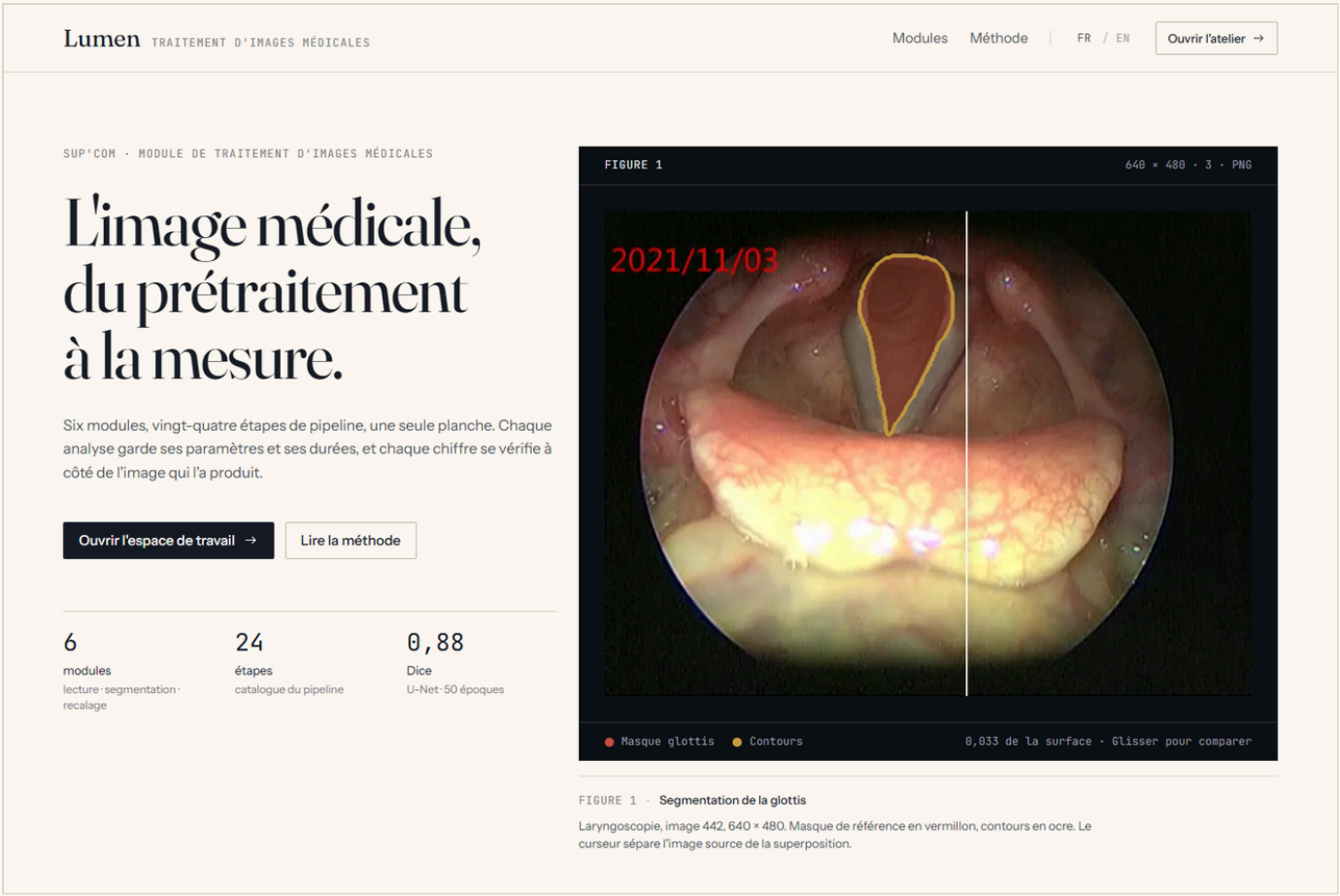


FIGURE 2 Page d'accueil, 1440 px de large : la promesse en deux phrases, puis l'espace de travail montré avec une image du cours et son masque.

La direction visuelle tient en trois règles. Le papier ivoire porte la page, la scène sombre porte l'image, et un seul vermillon par écran signale l'action qui compte, ici le bouton qui lance l'analyse.

Les titres sont en Fraunces, le texte en Instrument Sans, les chiffres en JetBrains Mono à chasse fixe : les colonnes de mesures s'alignent d'elles-mêmes et les unités restent lisibles.

Le canvas est le cœur de l'espace de travail. Il dimensionne son tampon sur le facteur de densité de l'écran, puis dessine l'image et le masque dans le même repère.

IMAGECANVAS.TSX, TAMPON AU FACTEUR DPR

```
    if (canvas.width !== Math.round(width * dpr) || canvas.height !== Math.round(height * dpr)) {
      canvas.width = Math.round(width * dpr);
      canvas.height = Math.round(height * dpr);
    }
    sizeRef.current = { width, height, dpr };

    const ctx = canvas.getContext("2d");
    if (!ctx) return;
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.clearRect(0, 0, width, height);
```

Le masque n'est pas une image bitmap posée par-dessus : il est teinté une fois dans un canvas hors écran, puis posé avec une opacité réglable, ce qui laisse voir l'image dessous.

IMAGECANVAS.TSX, POSE DU CALQUE DE MASQUE

```
      if (current.maskVisible && tintRef.current) {
        ctx.globalAlpha = current.opacity;
        ctx.drawImage(tintRef.current.canvas, -imageWidth / 2, -imageHeight / 2, imageWidth, imageHeight);
        ctx.globalAlpha = 1;
```

5 Résultats mesurés

Les cinq chaînes prêtes à l'emploi ont été rejouées sur les démonstrations de l'API, avec le masque de référence fourni. Les scores ci-dessous sortent de la réponse de `POST /api/analyses`, pas d'un calcul à part.

TABLEAU 4. CINQ CHAÎNES PRÊTES À L'EMPLOI, IMAGE DU COURS SAUF LE U-NET

CHAÎNE	DICE	IOU	DURÉE
Seuillage d'Otsu inversé	0,086	0,045	31 ms
k-means 3 classes	0,461	0,300	707 ms
Watershed sur le gradient	0,907	0,829	39 ms
Contours actifs Chan-Vese	0,484	0,319	1804 ms
U-Net endoscopie	0,959	0,921	76 ms

Le U-Net est mesuré sur une image de son jeu d'entraînement, donc dans son domaine : 67 ms d'inférence pour 0,959 de Dice, chiffre optimiste puisque la page d'accueil annonce 0,88. Les quatre autres chaînes travaillent sur l'image du cours.

Le temps part dans l'algorithme lui-même : 684 ms sur 707 pour l'étape k-means, 1800 ms sur 1804 pour Chan-Vese, 22 ms sur 39 pour le watershed. Les post-traitements restent sous 5 ms.

TABLEAU 5. RECALAGE DE LA PAIRE FIRE A01, DIX POINTS DE CONTRÔLE

MÉTHODE	ERREUR MOYENNE	SUCCÈS À 10 PX	NCC
Homographie SIFT et MAGSAC	2,87 px	100 %	0,937
Affine	1,24 px	100 %	0,944
Flot optique global	1,33 px	100 %	0,945

Sur cette paire, l'affine est la plus proche : l'écart entre les deux rétines est surtout une rotation et une translation. L'homographie reste la méthode par défaut, parce qu'elle couvre les déformations projectives du reste du jeu.

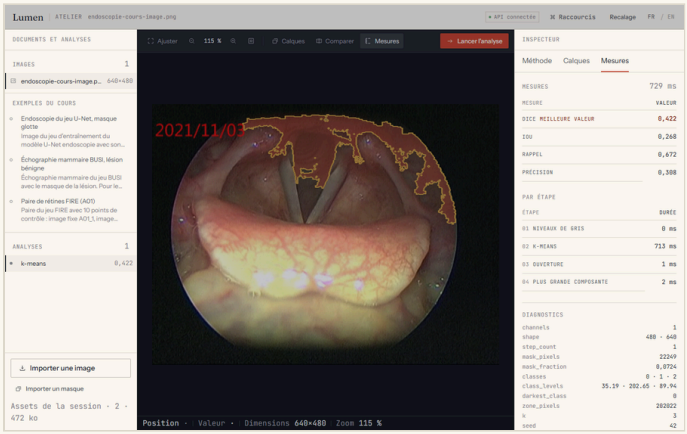


FIGURE 3A k-means réglé à la main : 0,422 et 729 ms, dont 713 ms pour l'étape k-means.

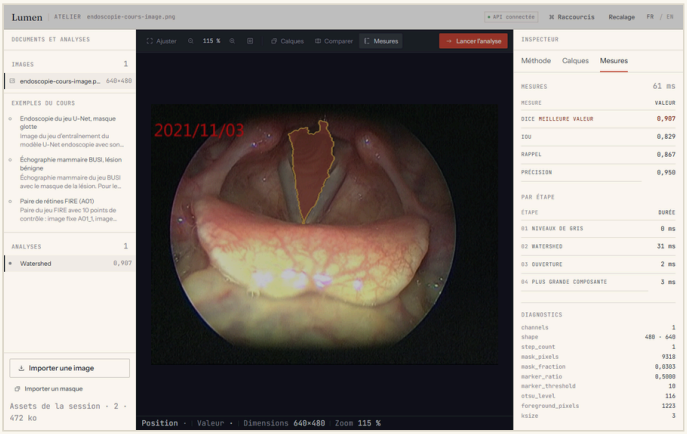


FIGURE 3B Chaîne watershed : 0,907, 61 ms au total, dont 31 ms pour le watershed.

LECTURE DES RÉSULTATS

Le watershed et le U-Net dominent, à 0,907 et 0,959 de Dice : la forme de la glotte ressort des deux côtés.

Otsu seul échoue à 0,086 : la classe sombre du fond occupe presque toute l'image, donc le masque attrape le contour du champ de vision au lieu de la glotte.

Entre les deux, k-means et Chan-Vese restent autour de 0,46 et 0,48, utiles pour montrer ce que donne une méthode nue.

D'où les chaînes prêtes à l'emploi : un prétraitement, un post-traitement et une règle de choix de composante font plus que l'algorithme seul.

6 Recalage

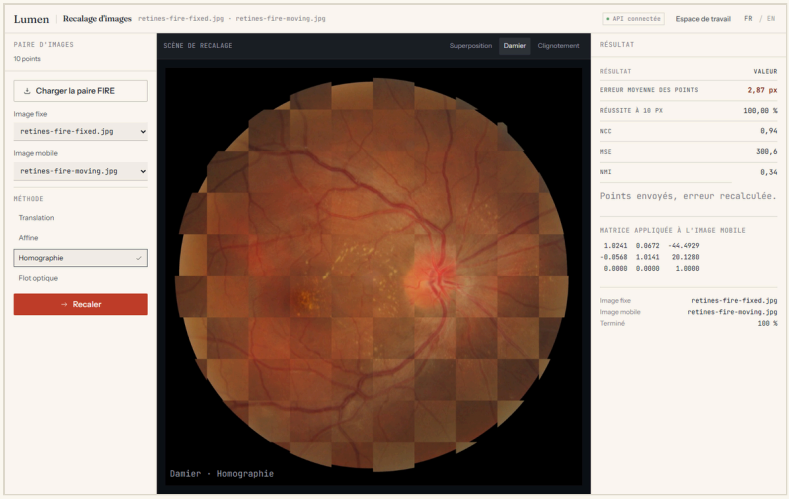


FIGURE 4 Page de recalage sur la paire FIRE A01 : vue en damier, tableau des métriques et matrice 3 x 3.

Le damier montre que les vaisseaux se raccordent de part et d'autre des cases : l'homographie ramène l'image mobile dans le repère de la fixe avec 2,87 px d'erreur moyenne sur les dix points.

Les dix points tombent tous sous 10 px, soit 100 pour cent de succès, et le NCC monte à 0,937 après recalage. Le package, mesuré sur sa propre paire de test à résolution doublée, donne 3,2 px pour la même méthode : les deux valeurs sont du même ordre.

La matrice s'affiche en clair sous le damier : rotation de 3,2 degrés, translation de 44 px en x et 20 px en y. Sa dernière ligne vaut 0,0000 0,0000 1,0000 : la déformation projective reste négligeable sur cette paire.

7 Choix d'ingénierie et limites

Quatre décisions qui ont compté

- **Proxy de même origine.** Le cookie de session est httpOnly et le canvas relit les pixels du masque : les deux exigent une seule origine. Une réécriture Next.js vers `/lum-api` règle les deux cas sans ouvrir CORS.
- **ONNX sur processeur, pas de serveur GPU.** Les deux U-Net pèsent 7,8 Mo chacun et l'inférence 256 x 256 prend 67 ms. Louer un GPU pour cela coûterait plus cher que le temps gagné.
- **File de tâches interne plutôt que Redis.** Un pool de threads dans le processus suffit pour une soutenance. Il tient la progression en base et la pousse en SSE, sans service supplémentaire à démarrer.
- **Presets plutôt que paramètres nus.** Les cinq chaînes portent leurs scores attendus, donc l'utilisateur compare des résultats reproductibles au lieu de régler un seuil à l'aveugle.

Trois limites assumées

- Pas de comptes utilisateurs : la session est un cookie anonyme, suffisant pour un atelier, insuffisant pour un usage réel.
- SQLite et stockage local ne sont pas distribués : deux instances de l'API ne partagent ni la base ni les fichiers.
- Le U-Net endoscopie ne connaît que son domaine : sur un autre cadrage il rend un masque vide, ce qui se voit à l'écran et doit être dit avant de conclure à un bug.

La suite est écrite dans `docs/DEPLOY.md` : interface sur Vercel, API sur Railway avec un volume pour `storage/` et la base, variables `LUMEN_*` posées au démarrage, vérification sur `/api/health`.

8 Conclusion

- La plateforme couvre les six ateliers du module dans une seule interface, du prétraitement à la comparaison chiffrée.
- Les scores obtenus reproduisent ceux du cours : 0,086 pour Otsu, 0,461 pour k-means, 0,907 pour watershed, 0,959 pour le U-Net.
- Le recalage de la paire FIRE tombe à 2,87 px d'erreur moyenne, avec 100 pour cent des points sous 10 px et un NCC de 0,937.
- Le code reste séparé en trois couches testables, ce qui permet de changer une méthode sans toucher ni à l'API ni à l'interface.